

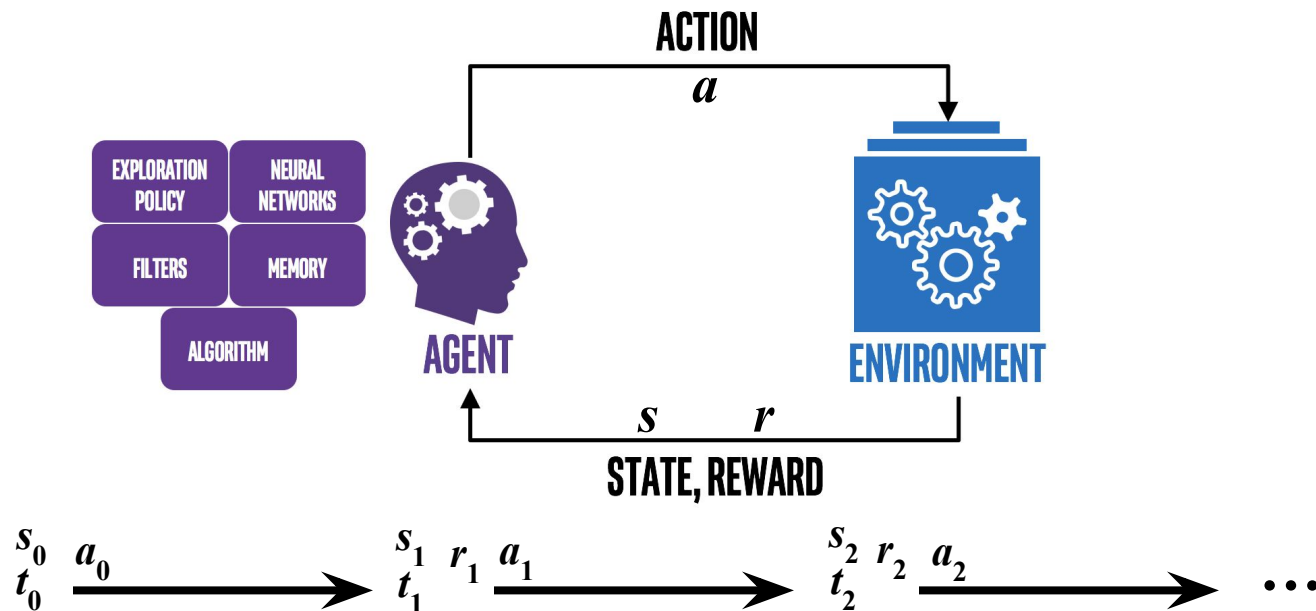


Aprendizado por Reforço

Aulas 5 e 6: Processo de Decisão de Markov

Prof. Charles Madeira

Bases do Aprendizado por Reforço



- O agente aprende a se aproximar de uma estratégia comportamental ótima a partir da interação repetitiva com o ambiente
- O ambiente pode ser estocástico e desconhecido (sem definição do modelo de regras de transição)

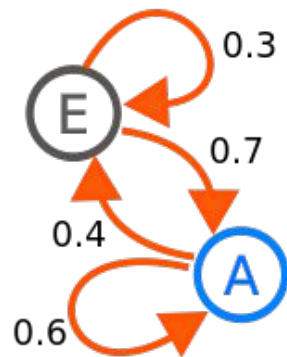
Bases do Aprendizado por Reforço

- Existe algumas teorias que agrupam diversos aspectos úteis para a formalização do Aprendizado por Reforço
 - Teoria das probabilidades
 - Representação e evolução dos estados do ambiente a partir do formalismo das cadeias de Markov
 - Teoria da decisão
 - Problemática fundamental da tomada de decisão de um agente
 - Teoria da utilidade
 - Avaliação do valor de uma decisão (referente ao desempenho)
- As formalizações matemáticas do Aprendizado por Reforço são construídas sob a família de **Processos de Markov**
 - Considera que o ambiente pode ser observado
 - As situações observadas se referem aos estados reais do ambiente



Cadeia de Markov

- Descreve um sistema que pode transitar entre estados de acordo com leis de dinâmica, que geralmente são desconhecidas
- O observador não pode influenciar o sistema, mas apenas observar as mudanças de estado
 - **Conjunto finito de estados** (S): todos os estados possíveis em que o ambiente pode se encontrar
 - **Função de Transição** (T): matriz quadrada na qual cada célula (i, j) contém a probabilidade de transição do estado i para o estado j
 - **Propriedade de Markov**: o futuro do ambiente, a partir de qualquer estado, depende apenas do próprio estado, e não de toda a história de estados anteriores
 - Tem como principal característica o fato de tornar cada estado observável “autocontido” para descrever a dinâmica futura do ambiente
 - **Estacionaridade**: propriedade de Markov implica estacionaridade, que consiste em não mudar a distribuição de transição subjacente ao longo do tempo para qualquer estado



Processo de Recompensa de Markov

- Estende a cadeia de Markov adicionando a noção de **recompensa** e um **fator de desconto**
- **Recompensa** (r): para cada transição de estado, atribui um valor escalar (positivo ou negativo)
 - Pode ser representada por uma matriz similar à de transição
- **Fator de Desconto** (γ): valor entre 0 e 1 que determina o quão “longe no futuro” o agente considera as recompensas ao estimar o retorno
 - Se $\gamma = 1$: o agente tem visibilidade perfeita de todas as recompensas futuras
 - Se $\gamma = 0$: o agente é “de vista curta”, preocupando-se apenas com a recompensa imediata
 - Se $\gamma > 0$ e $\gamma < 1$: o agente considera recompensas futuras, mas com um peso decrescente
- **Retorno** (R_t): Para cada ponto no tempo t , o retorno é a soma das recompensas subsequentes, em que recompensas mais distantes são multiplicadas pelo fator de desconto elevado à potência do número de passos

$$R_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots = \sum_{k=0}^T \gamma^k r_{t+k+1}, \text{ où } 0 \leq \gamma \leq 1$$

Processo de Recompensa de Markov

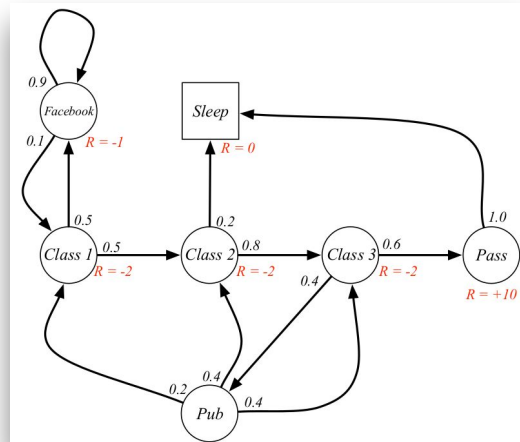
- **Valor do Estado** ($V(s)$): expectativa matemática do retorno para qualquer estado s , que consiste no retorno médio (ou esperado) que se obtém ao seguir o processo de recompensa de Markov a partir desse estado

$$V^\pi(s) = E_\pi \{R_t | s_t = s\} = E_\pi \left\{ \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} | s_t = s \right\}$$

- O valor do estado $V(s)$ é crucial para o aprendizado de uma **política de ação** π que permita maximizar o retorno R_t de recompensa cumulativa a longo prazo, indo de um estado inicial até um estado final
- O fator de desconto γ é crucial para lidar com processos que podem ter um número infinito de transições, evitando valores infinitos para o retorno R_t e, conseqüentemente, limitando o horizonte de cálculo

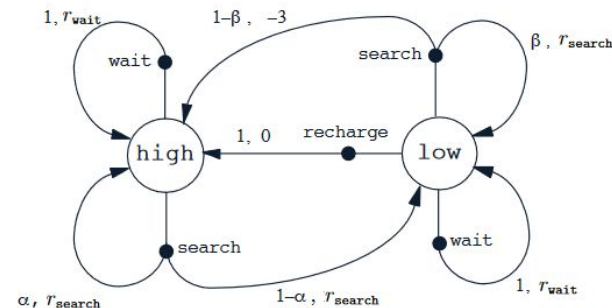
Processo de Decisão de Markov (MDP)

- Formalização teórica mais completa para problemas de Aprendizado por Reforço, adicionando a capacidade do agente tomar ações ao processo de recompensa de Markov
- MDP é definido por uma tupla (S, A, T, R)
 - **S** é um conjunto finito de estados (com estado inicial s_0)
 - **A** é um conjunto finito de ações aplicáveis a cada estado **s**
 - **T**: $S \times A \times S \rightarrow [0,1]$ é uma função de probabilidade de transição
 - $T(s, a, s') = P \{s_{t+1} = s' \mid s_t = s, a_t = a\}$
 - **R**: $S \times A \times S \rightarrow \mathbb{R}$ é uma função de recompensa de valores reais
 - $R(s, a, s') = E \{r_{t+1} \mid s_t = s, a_t = a, s_{t+1} = s'\}$
- A função **T** representa o modelo do ambiente e está sujeita à **propriedade de Markov**
 - É possível determinar o próximo estado **s'** considerando apenas o estado atual **s** e a ação **a**
 - $P \{s_{t+1} = s' \mid s_t, a_t\} = P \{s_{t+1} = s' \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0\}$



Modelagem de um robô coletor em MDP

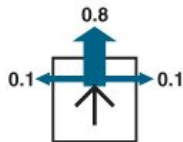
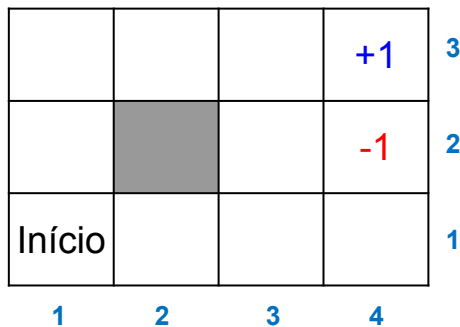
- MDP definido pela tupla (S, A, T, R)
 - Espaço de estados S do ambiente (níveis da bateria recarregável)
 - {bateria carregada (high), bateria descarregada (low)}
 - Espaço de ações A do agente
 - {search, wait, recharge}
 - Função de probabilidade de transição $T(s, a, s')$
 - Uma ação de busca leva o agente a gastar bateria até que passe a necessitar de recarga para poder funcionar
 - Uma ação de espera não gasta bateria
 - Uma ação de recarga leva o agente a estar apto novamente a coletar
 - Função de recompensa $R(s, a, s')$
 - Positiva quando o robô realiza atividade de coleta
 - Nula quando o robô está carregando a bateria



s	a	s'	$p(s' s, a)$	$r(s, a, s')$
high	search	high	α	r_{search}
high	search	low	$1 - \alpha$	r_{search}
low	search	high	$1 - \beta$	-3
low	search	low	β	r_{search}
high	wait	high	1	r_{wait}
high	wait	low	0	r_{wait}
low	wait	high	0	r_{wait}
low	wait	low	1	r_{wait}
low	recharge	high	1	0
low	recharge	low	0	0.

Modelagem de um Grid 4 x 3 em MDP

- Espaço de estados **S** do ambiente (células nas quais o agente pode estar localizado no grid)
 - $\{(1,1), (1,2), (1,3), (2,1), (2,3), (3,1), (3,2), (3,3), (4,1), (4,2), (4,3)\}$
- Espaço de ações **A** do agente
 - {Acima, Abaixo, Esquerda, Direita}
- Função de probabilidade de transição **T(s, a, s')**
 - Cada ação pode levar o agente a alcançar o efeito pretendido (probabilidade 0,8)
 - Cada ação pode levar o agente para alguma direção perpendicular do efeito pretendido (probabilidade 0,1 para cada uma delas)
 - Se o agente for para fora do grid, ele permanecerá na mesma célula
 - Atinge o estado final quando o agente alcança a célula (4,2) ou (4,3)
- Função de recompensa **R(s, a, s')**
 - +1 quando o agente atinge a célula (4,3)
 - -1 quando o agente atinge a célula (4,2)
 - -0,04 para todas as outras transições de estado



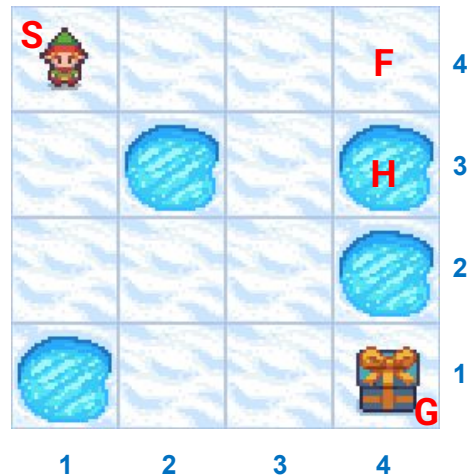
Exemplo do Frozen Lake

- Ambiente da série *Toy Text* do *Gymnasium*, que consiste em um problema clássico de aprendizado por reforço
 - Grid referente a um lago congelado no qual um personagem pode se deslocar na horizontal e na vertical
- Objetivo:
 - Caminhar no lago congelado, partindo do estado inicial (S) até chegar ao estado objetivo (G)
 - Caminhar apenas em ladrilhos congelados (F), evitando os buracos (H)
- Nem sempre o personagem poderá se mover na direção pretendida devido à natureza escorregadia do lago congelado (ambiente estocástico)



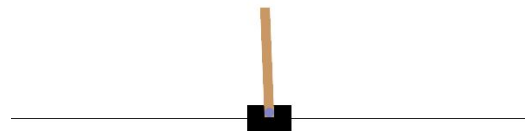
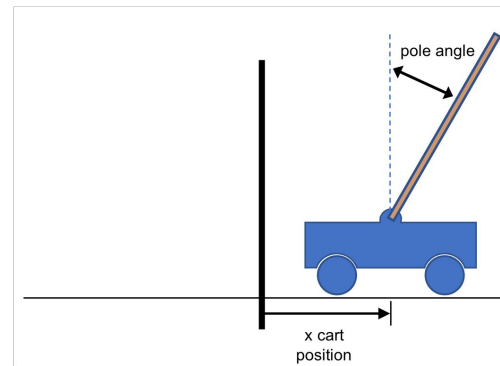
Modelagem do Frozen Lake em MDP

- Espaço de estados **S** do ambiente (células nas quais o agente pode estar localizado em um grid N x N)
 - $\{(1,1), (1,2), \dots, (1,N), (2,1), (2,2), \dots, (2,N), \dots, (N,1), (N,2), \dots, (N,N)\}$
- Espaço de ações **A** do agente
 - {Acima, Abaixo, Esquerda, Direita}
- Função de probabilidade de transição **T(s, a, s')**
 - Quando o lago não está escorregadio (determinístico), o agente avança na direção pretendida (probabilidade 1,0)
 - Quando o lago está escorregadio (estocástico), o agente avança:
 - na direção pretendida (probabilidade $\frac{1}{3}$)
 - em qualquer direção perpendicular (probabilidade $\frac{1}{3}$ para cada)
 - Se o agente for para fora do grid, ele permanecerá na mesma célula
 - Atinge o estado final quando o agente alcança a célula (N,N) ou cai em um buraco
- Função de recompensa **R(s, a, s')**
 - +1 quando o agente atinge a célula (N,N)
 - Nulo para todas as outras transições de estado



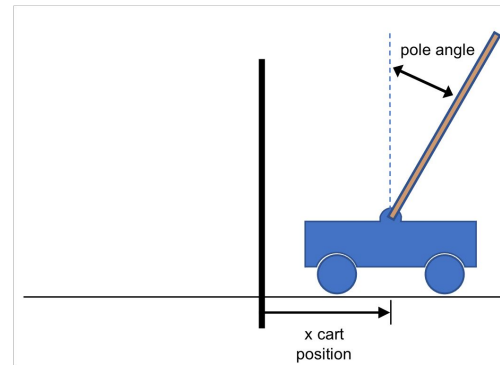
Exemplo do Cart Pole

- Ambiente da série *Classic Control do Gymnasium*, que consiste em um problema clássico de aprendizado por reforço
 - Poste preso a um carrinho por uma junta em formato de pêndulo
 - Carrinho move-se ao longo de um trilho sem atrito
 - Aplica-se forças no carrinho nas direções da esquerda e direita
- Objetivo
 - Equilibrar o poste na posição vertical pelo maior tempo possível
- O espaço de estados é contínuo e, portanto, muito grande
 - Torna-se bem mais difícil aplicar métodos tabulares para calcular a função valor
 - Abordagens de aproximação de função se mostram como excelentes alternativas



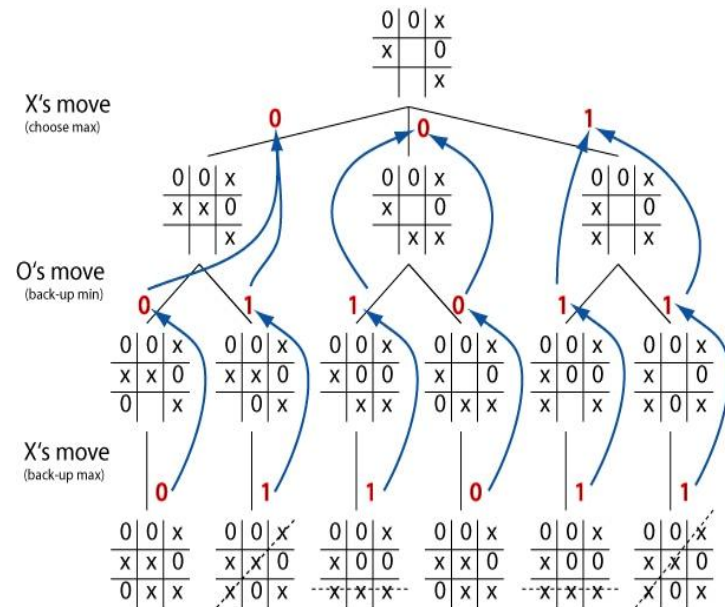
Modelagem do Cart Pole em MDP

- Espaço de estados **S** do ambiente = vetor de 4 variáveis
 - Posição do carrinho: $[-4.8 \dots +4.8]$
 - Velocidade do carrinho: $[-\text{Inf} \dots +\text{Inf}]$
 - Ângulo do pêndulo: $[-24^\circ \dots +24^\circ]$
 - Velocidade angular do pêndulo: $[-\text{Inf} \dots +\text{Inf}]$
 - Nota: o episódio termina nas seguintes situações:
 - Posição do carrinho fora do intervalo ± 2.4
 - Ângulo do pêndulo fora do intervalo $\pm 12^\circ$ ($\pm 0.2095 \text{ rad}$)
 - Duração do episódio maior que 500
- Espaço de ações **A** do agente = $\{0, 1\}$ indica a direção da força fixa em que o carrinho é empurrado
 - 0: Empurra o carrinho para a esquerda
 - 1: Empurra o carrinho para a direita
 - Notas:
 - Velocidade varia de acordo com o ângulo que o poste está apontando
 - Centro de gravidade do poste varia a quantidade de energia necessária para mover o carrinho
- Função de probabilidade de transição **T**
 - ???
- Função de reforço **R**
 - +1: cada passo dado (limite de 475)



Exercício: modelagem do Tic-Tac-Toe (jogo da velha)

- Espaço de estados **S** do ambiente (configurações possíveis de um grid 3 x 3)
 - $\{(\emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset), \dots, (0, 0, X, X, \emptyset, 0, \emptyset, \emptyset, X), \dots, (0, 0, X, X, X, 0, X, 0, X)\}$
estado inicial estado intermediário estado final
- Espaço de ações **A** do agente (considerando jogar **X**)
 - Inserir **X** em alguma das células do grid
- Função de probabilidade de transição **T(s, a, s')**
 - ???
 - Obs: as regras do jogo são claras e objetivas, mas a estratégia do adversário pode variar
- Função de recompensa **R(s, a, s')**
 - ???
 - Obs: o quanto cada transição de estado leva o agente para mais próximo de alguma configuração que caracterize fim de partida com resultado positivo?



Política π

- Como deve ser uma solução para um problema modelado por meio de um MDP?
 - Qualquer sequência fixa de ações não resolverá o problema porque o agente pode acabar em um estado diferente do objetivo
 - Uma solução tem de especificar o que o agente deve fazer para qualquer estado alcançado
- Uma solução desse tipo é chamada de **política π** , com $\pi(s)$ sendo a associação da ação recomendada para o estado s
- Se o agente tiver uma **política completa**, não importará o resultado de uma ação a específica, pois ele sempre saberá o que fazer em seguida

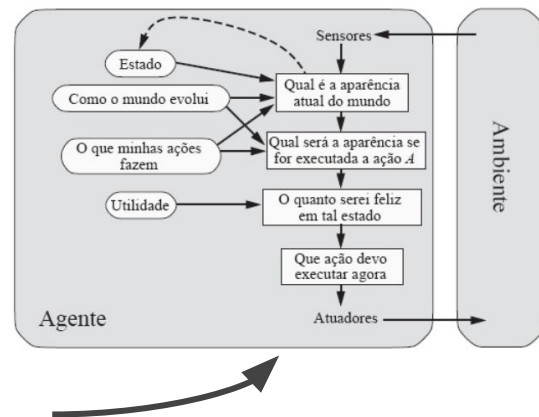
?	?	?	+1	3
?		?	-1	2
? Início	?	?	?	1
1	2	3	4	



Política ótima π^*

- Toda vez que uma dada política for executada a partir do estado inicial, a natureza estocástica do ambiente poderá levar a um histórico diferente
- A qualidade de uma política é, portanto, medida pela utilidade esperada dos históricos de estados possíveis gerados por essa política
- Uma **política ótima π^*** é aquela que produz a utilidade esperada mais alta, em que o agente decide o que fazer consultando o seu estado atual **s** e executando a ação **$\pi^*(s)$**
- Política representa explicitamente a **função do agente**
 - É uma descrição de um agente reflexivo simples, calculada a partir das informações usadas por um **agente baseado em utilidade**

?	?	?	+1	3
?		?	-1	2
? Início	?	?	?	1
1	2	3	4	



Política ótima π^*

- Existem duas políticas ótimas para o grid 4 x 3 porque o agente não faz diferença entre ir para a esquerda e ir para cima a partir de (3,1)
 - Ir para a esquerda é mais seguro, porém mais longo (política conservadora)
 - Ir para cima é mais rápido, mas corre o risco de parar em (4,2) por acidente (política arriscada)
- O equilíbrio entre risco e recompensa muda dependendo do valor de $r = R(s, a, s')$ para as transições entre estados não terminais

→	→	→	+1	3
↑		↑	-1	2
↑	←	↖	←	1
1	2	3	4	

Política ótima π^*

- Abaixo são mostradas políticas ótimas para quatro intervalos diferentes de r
 - Se $r < -1,6497$, a vida é tão difícil que o agente vai direto para a saída mais próxima, ainda que a saída tenha o valor -1
 - Se $-0,7311 < r < -0,4526$, a vida é bastante desagradável, logo o agente toma a rota mais curta até o estado $+1$ a partir de (2,1), (3,1) e (3,2), mas a partir de (4,1) o custo de alcançar $+1$ é tão alto que o agente prefere ir para -1
 - Se a vida é apenas ligeiramente ruim ($-0,0274 < r < 0$), a política ótima não assume absolutamente nenhum risco. Em (4,1) e (3,2), o agente segue diretamente para fora do estado -1 , de forma que não possa cair nesse estado por acidente, embora isso signifique tentar ir para fora do grid várias vezes
 - Se $r > 0$, a vida é positivamente agradável e o agente evita ambas as saídas. Desde que as ações em (4,1), (3,2) e (3,3) sejam as que estão representadas, toda política é ótima, e o agente obtém recompensa total infinita porque nunca alcançará um estado terminal

→	→	→	+1
↑		→	-1
→	→	→	↑
1	2	3	4

$r < -1,6497$

→	→	→	+1
↑		↑	-1
↑	→	↑	↑
1	2	3	4

$-0,7311 < r < -0,4526$

→	→	→	+1
↑		←	-1
↑	←	←	↓
1	2	3	4

$-0,0274 < r < 0$

↕	↕	←	+1
↕		←	-1
↕	↕	↕	↓
1	2	3	4

$r > 0$

Utilidades ao longo do tempo

- O desempenho do agente é medido por uma **soma de recompensas** para os estados visitados
- Essa escolha de medida de desempenho não é arbitrária, mas não é a única possibilidade da função utilidade sobre históricos de ambiente
 - Teoria da utilidade de multiatributo:
$$V_h([s_0, a_0, s_1, a_1, \dots, s_n])$$
- O horizonte para a tomada de decisão pode ser **finito** (tempo fixo N) ou **infinito** (sem limite de tempo)

→	→	→	+1	3
↑		↑	-1	2
↑	←	↖	←	1
1	2	3	4	

Utilidades ao longo do tempo

- **Horizonte finito:**

$$V_h([s_0, a_0, s_1, a_1, \dots, s_{N+k}]) = V_h([s_0, a_0, s_1, a_1, \dots, s_N])$$

para todo $k > 0$

- Exemplo do agente começando na célula (3,1)

- Se $N = 3$, para ter qualquer chance de alcançar o estado com recompensa **+1**, o agente deve ir diretamente para ele, com a ação ótima sendo de ir **Acima**
- Se $N = 100$, o agente tem bastante tempo para seguir a rota segura, dirigindo-se para a **Esquerda**

→	→	→	+1	3
↑		↑	-1	2
↑	←	↖	←	1
1	2	3	4	

Utilidades ao longo do tempo

- Com um horizonte finito, a ação ótima em um dado estado pode mudar com o passar do tempo
 - A **política ótima** é **não estacionária**
- Por outro lado, sem um limite de tempo fixo, não existe nenhuma razão para se comportar de maneira diferente no mesmo estado em momentos distintos
- Consequentemente, a ação ótima depende apenas do estado atual
 - A **política ótima** é **estacionária**
- Políticas para o caso de horizonte infinito são mais simples que aquelas para o caso de horizonte finito



Tipos de recompensas

- O caráter estacionário é uma hipótese de aparência bastante inócua, mas com consequências muito fortes, que dispõe de duas maneiras de atribuir utilidades às sequências
- Recompensas aditivas
 - $V_h([s_0, a_0, s_1, a_1, s_2, \dots]) = R(s_0, a_0, s_1) + R(s_1, a_1, s_2) + R(s_2, a_2, s_3) + \dots$
- Recompensas aditivas descontadas
 - $V_h([s_0, a_0, s_1, a_1, s_2, \dots]) = R(s_0, a_0, s_1) + \gamma R(s_1, a_1, s_2) + \gamma^2 R(s_2, a_2, s_3) + \dots$
 - Conforme visto anteriormente, o fator de desconto γ de valor $[0,1]$ que descreve a preferência de um agente por recompensas atuais sobre recompensas futuras
 - Quando γ é próximo de 0, as recompensas no futuro distante são vistas como insignificantes
 - Quando γ é 1, recompensas descontadas são exatamente equivalentes a recompensas aditivas, logo recompensas aditivas constituem um caso especial de recompensas descontadas



Utilidades ao longo do tempo

- Há vários motivos para recompensas aditivas descontadas fazerem sentido:
 - 1) **Empírico**, pois tanto seres humanos quanto animais parecem valorizar mais as recompensas a curto prazo do que as recompensas no futuro distante
 - 2) **Econômico**, pois se as recompensas são monetárias, então realmente é melhor obtê-las mais cedo, uma vez que podem ser investidas e produzir retornos
 - Equivale a uma taxa de juros $(1 / \gamma) - 1$
 - 3) **Incerteza** sobre as verdadeiras recompensas, pois elas podem nunca chegar por diversos motivos que não são levados em conta no modelo de transição
 - Equivale a adicionar uma probabilidade $1 - \gamma$ de término acidental

Utilidades ao longo do tempo

- Há vários motivos para recompensas aditivas descontadas fazerem sentido:
 - 4) **Propriedade natural** de preferências sobre históricos, na qual na terminologia da teoria de utilidade multiatributo, cada transição de estado (s_t, a_t, s_{t+1}) pode ser vista como um atributo do histórico $[s_0, a_0, s_1, a_1, s_2, \dots]$
 - A função utilidade pode depender dos atributos de maneiras arbitrariamente complexas
 - Uma suposição de independência de preferência altamente plausível pode ser feita, em que as preferências do agente são **estacionárias**
 - Se duas sequências de estados $[s_0, a_0, s_1, a_1, s_2, \dots]$ e $[s'_0, a'_0, s'_1, a'_1, s'_2, \dots]$ começam com o mesmo estado, então as duas sequências devem ser ordenadas por preferência, do mesmo modo que as sequências $[s_1, a_1, s_2, \dots]$ e $[s'_1, a'_1, s'_2, \dots]$
 - Isso significa que, se preferir um futuro em detrimento de outro que começará amanhã, o agente ainda deverá preferir esse futuro se tiver de começar hoje

Utilidades ao longo do tempo

- Há vários motivos para recompensas aditivas descontadas fazerem sentido:
 - 5) **Conveniência**, pois faz desaparecer alguns infinitos desagradáveis
 - Se o ambiente não contiver estados terminais ou o agente nunca alcançar esses estados, todos os históricos serão infinitamente longos e as recompensas aditivas serão infinitas
 - Mas se o ambiente contiver estados terminais e o agente oferecer a garantia de chegar a algum deles (**política própria**), nunca será preciso comparar sequências infinitas
 - No grid 4 x 3, a **política é imprópria** se a recompensa para os estados não terminais for positiva ($r > 0$), afastando o agente dos estados terminais por ter recompensa total infinita, levando os algoritmos clássicos para resolução de PDM a falhar
 - Logo, com recompensas descontadas, a utilidade de uma sequência infinita é **finita**, com as recompensas sendo limitadas por $\pm R_{\max}$

$$V_h([s_0, a_0, s_1, \dots]) = \sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1}) \leq \sum_{t=0}^{\infty} \gamma^t R_{\max} = \frac{R_{\max}}{1-\gamma}$$

Políticas ótimas e utilidades dos estados

- Com a utilidade de uma determinada sequência de estados sendo a soma das recompensas descontadas, podemos comparar políticas por meio das utilidades esperadas obtidas
- A **utilidade esperada** é obtida executando π a partir de s , tendo a esperança E relacionada à distribuição de probabilidade sobre sequências de estados determinadas por s e π

$$V^{\pi}(s) = E \left[\sum_{t=0}^{\infty} \gamma^t R(S_t, \pi(S_t), S_{t+1}) \right]$$

- Em todas as políticas que o agente poderia escolher executar a partir de s , uma (ou mais) terá utilidades esperadas maiores do que todas as outras

$$\pi_s^* = \operatorname{argmax}_{\pi} V^{\pi}(s)$$

Políticas ótimas e utilidades dos estados

- Assim, $V^{\pi^*}(s)$ é a utilidade verdadeira de um estado, ou seja, a soma esperada de recompensas descontadas se o agente executar uma política ótima
- A função utilidade $V(s)$ permite ao agente selecionar ações usando o princípio da **utilidade esperada máxima**
 - Escolher a ação que maximiza a **recompensa esperada para a próxima transição**, associada à **utilidade descontada esperada do próximo estado**

$$\pi^*(s) = \operatorname{argmax}_{a \in A(s)} \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V(s')]$$

Equação de Bellman

- Assim, a utilidade $V(s)$ de um estado s é definida pela soma esperada de recompensas descontadas a partir do instante atual, conhecida como **equação de Bellman**
 - A utilidade de um estado é a **recompensa esperada para a próxima transição**, somada à **utilidade descontada do próximo estado**, supondo que o agente escolha a ação ótima

$$V(s) = \max_{a \in A(s)} \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V(s')]$$

- Aplicando Bellman, obtemos as utilidades dos estados do grid 4 x 3 (ex.: a partir da célula (1, 1) com $\gamma = 1$ e $r = -0,04$)

$$V(1,1) = \max \left\{ \begin{aligned} &[0.8(-0.04 + \gamma V(1,2)) + 0.1(-0.04 + \gamma V(2,1)) + 0.1(-0.04 + \gamma V(1,1))], \\ &[0.9(-0.04 + \gamma V(1,1)) + 0.1(-0.04 + \gamma V(1,2))], \\ &[0.9(-0.04 + \gamma V(1,1)) + 0.1(-0.04 + \gamma V(2,1))], \\ &[0.8(-0.04 + \gamma V(2,1)) + 0.1(-0.04 + \gamma V(1,2)) + 0.1(-0.04 + \gamma V(1,1))] \end{aligned} \right\}$$

Acima
 Esquerda
 Abaixo
 Direita

Utilidade de cada estado

0,8516	0,9078	0,9578	+1	3
0,8016		0,7003	-1	2
0,7453	0,6953	0,6514	0,4279	1
1	2	3	4	

Política derivada da função utilidade

Política ótima

→	→	→	+1	3
↑		↑	-1	2
↑	←	↖	←	1
1	2	3	4	

Função utilidade-ação

- Outro elemento importante é a **função utilidade-ação**, ou função **$Q(s, a)$** , que é a utilidade esperada de seguir determinada ação em um determinado estado

$$V(s) = \max_a Q(s, a)$$

- A política ótima pode ser extraída a partir da função Q

$$\pi^*(s) = \operatorname{argmax}_a Q(s, a)$$

- Também podemos desenvolver uma equação de Bellman para funções Q , notando que a recompensa total esperada para seguir uma ação é sua recompensa imediata, associada à utilidade descontada do estado resultante

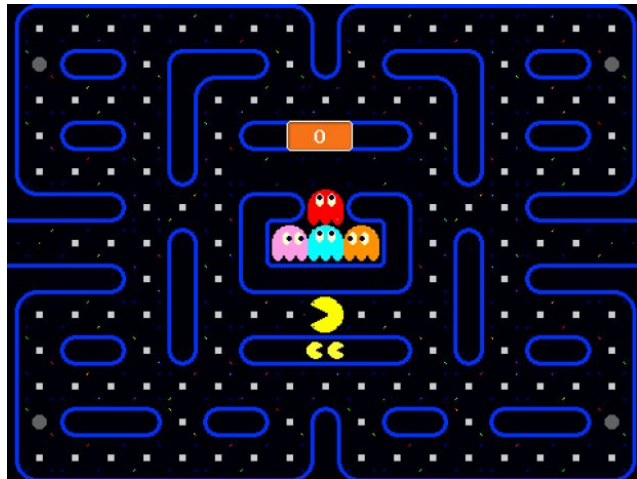
$$\begin{aligned} Q(s, a) &= \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V(s')] \\ &= \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma \max_{a'} Q(s', a')] \end{aligned}$$

Algoritmos para MDP

- As soluções das equações de Bellman para V ou para Q oferecem o que é necessário para encontrar uma política ótima, visando solucionar MDP
- Existem diversos algoritmos para tal, alguns que permitem gerar **soluções exatas offline** e outros que permitem gerar **soluções aproximadas online**
- Algoritmos para soluções exatas offline
 - Iteração de Valor
 - Iteração de Política
 - Programação Linear
- Algoritmo para soluções aproximadas online
 - Planejamento de Monte Carlo

Exercício: Pac-Man em formato de MDP

- Ambiente consiste em um labirinto no qual o jogador deve controlar um personagem que coleta pastilhas e foge de fantasmas que também se deslocam no cenário
- O objetivo é coletar todas as pastilhas sem ser atingido pelos fantasmas



Exercício: Xadrez em formato de MDP

- Ambiente consiste em um tabuleiro de 8x8 no qual cada jogador tem 16 peças: 1 rei, 1 dama, 2 torres, 2 bispos, 2 cavalos e 8 peões
- Cada peça é capaz de se movimentar de uma maneira específica
- O objetivo é dar xeque-mate no rei adversário, que acontece quando o rei está em uma posição na qual ele pode ser capturado (em xeque), sem chance de escapar
- **Dica:** Analisar a complexidade do problema

